Livrable L2 – Cahier expérimental

Allocation adaptative des ressources de calcul et de recherche documentaire pour des systèmes LLM locaux, économes et fiables

Benchmark, systèmes comparés, métriques, critères de succès et pilote

Alae Lahbichi – Étudiant en 5e année IA/Data

a.lahbichi2004@gmail.com

Dépôt GitHub : AlaeLahbichi/Adaptive-Compute-and-RAG-for-Efficient-Reliable-Local-LLMs

Version 1.0-pilote – 18 octobre 2026

Rôle du document. Ce cahier fixe, *avant* toute expérience finale, le benchmark, les systèmes comparés, les métriques, la méthode statistique et les critères de succès. Les paramètres sont consignés dans `configs/protocole_v1.json`. Après validation du pilote (section 9), le protocole sera gelé par l'étiquette Git `protocole-v1.0` : toute modification ultérieure devra être datée, justifiée et signalée dans l'article.

Résumé

Ce cahier expérimental définit le cadre d'évaluation d'une politique qui alloue, requête par requête, la taille du modèle, la recherche documentaire et l'abstention. Le benchmark réunit 630 questions en trois catégories (simple, complexe, non couverte), tirées de SQuAD 2.0 et HotpotQA, et un corpus de 3 293 passages. Il est séparé en un jeu de développement (180 questions) et un jeu de test (450 questions) réservé à l'évaluation finale. Deux modèles déjà présents sur la machine sont utilisés : Qwen2.5-1.5B et Qwen3-4B, quantifiés en 4 bits et exécutés avec MLX sur un Apple M3 Pro. Quatre stratégies fixes servent de référence. Le critère de succès principal est une réduction d'au moins 20 % de la latence moyenne par rapport à la stratégie « grand modèle + RAG », pour une perte de score global d'au plus 2 points. Un pilote sur le jeu de développement valide la chaîne complète et estime le temps de calcul.

Table des matières

1 Objet et principes **3**

2 Environnement matériel et logiciel **3**

3 Benchmark **3**
3.1 Sources . . . 3
3.2 Corpus documentaire . . . 4
3.3 Catégories de questions . . . 4
3.4 Taille et séparation . . . 4
3.5 Format d'une question . . . 4
3.6 Limites connues du benchmark . . . 4

4 Systèmes comparés **5**
4.1 Stratégies fixes (références) . . . 5
4.2 Références issues de la littérature (Partie III) . . . 5
4.3 Espace de décision de la méthode adaptative (Partie IV) . . . 5

5 Protocole d'exécution **5**

6 Métriques **6**

7 Analyse statistique **6**

8 Critères de succès (pré-enregistrés) **6**

9 Pilote **7**

10 Gel du protocole **8**

A Prompts **8**

B Reproduire **9**

1 Objet et principes

Le cahier des charges impose de *définir le succès avant les tests* et de *séparer développement et test*. Trois principes en découlent :

1. **Pré-enregistrement.** Les questions, les métriques, les systèmes de référence et les seuils de succès sont fixés ici avant de développer la méthode adaptative (Partie IV).
2. **Étanchéité du test.** Les seuils de la méthode sont réglés uniquement sur le jeu de développement. Le jeu de test n'est exécuté qu'une fois, sur la version finale, après le gel.
3. **Mêmes conditions pour tous.** Tous les systèmes sont évalués sur les mêmes questions, sur la même machine, avec les mêmes modèles, prompts et paramètres de génération.

2 Environnement matériel et logiciel

Élément	Valeur
Machine	Apple M3 Pro, 12 cœurs, 18 Go de mémoire unifiée, macOS 27.0.1
Exécution des modèles	MLX 0.32.3 et mlx-lm 0.32.0 (Python 3.12.11)
Petit modèle	`mlx-community/Qwen2.5-1.5B-Instruct-4bit` (environ 1,0 Go en mémoire)
Grand modèle	`mlx-community/Qwen3-4B-Instruct-2507-4bit` (environ 2,5 Go en mémoire)
Embeddings	`sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`
Recherche	Similarité cosinus exacte en NumPy, équivalente à un index FAISS `IndexFlatIP`
Énergie	`powermetrics` (macOS, droits administrateur), échantillonnage toutes les 200 ms

TABLE 1 – Environnement d'exécution. Les révisions exactes des modèles et les versions des paquets sont relevées automatiquement par `bench/check_env.py` à chaque série d'expériences.

Choix de MLX plutôt qu'Ollama. Le cahier des charges cite Ollama parmi les options possibles. MLX est retenu parce qu'il est optimisé pour Apple Silicon, que les modèles étaient déjà présents localement (aucun téléchargement supplémentaire) et qu'il donne accès au pic de mémoire consommé par le modèle. Les deux modèles tiennent ensemble en mémoire (environ 3,5 Go), ce qui permet de passer de l'un à l'autre sans rechargement.

Choix des modèles. Le ratio de taille est d'environ 2,7 (1,5 contre 4 milliards de paramètres). Les deux modèles appartiennent à deux générations de Qwen : l'écart de qualité ne s'explique donc pas uniquement par la taille. Ce point est sans conséquence pour l'objectif, qui est de disposer de deux niveaux de coût et de capacité, mais il sera mentionné comme limite. Llama-3.2-3B, également disponible, pourra servir de test de robustesse (QR4).

3 Benchmark

3.1 Sources

— **SQuAD 2.0**, jeu de développement (Rajpurkar et al., 2018) : 35 articles de Wikipédia (histoire, sciences, droit, géographie, institutions), avec des questions à réponse et des questions sans réponse rédigées pour ressembler aux premières.

— **HotpotQA**, jeu de développement en configuration *distractor* (Yang et al., 2018) : questions qui exigent de combiner deux pages Wikipédia, accompagnées de 8 paragraphes distracteurs.

Les deux jeux sont sous licence CC BY-SA 4.0. Leur empreinte SHA-256 est vérifiée à la construction.

3.2 Corpus documentaire

Le corpus est la « bibliothèque » dans laquelle la recherche documentaire puise ; il ne sert pas à entraîner les modèles. Il réunit les 1 204 paragraphes de SQuAD 2.0 et les 2 089 paragraphes distincts associés aux questions HotpotQA retenues, soit **3 293 passages**. Toutes les questions interrogent ce même corpus : la recherche doit donc retrouver les bons passages parmi plusieurs milliers.

3.3 Catégories de questions

Catégorie	Source	Définition	Attendu
Simple	SQuAD 2.0	La réponse se trouve dans un seul passage	Répondre
Complexe	HotpotQA	La réponse exige de combiner deux passages	Répondre
Non couverte	SQuAD 2.0	Le passage d'origine ne contient pas la réponse	S'abstenir
Ambiguë	Annotation manuelle	Plusieurs interprétations possibles	Préciser ou couvrir

Table 2 – Catégories du benchmark.

Catégorie « ambiguë ». Aucun des deux jeux ne fournit de questions ambiguës alignées sur le corpus. Elles seront rédigées à la main, avec un objectif de 50 questions, dans le fichier `data/benchmark/ambigues_a_annoter.jsonl`, qui contient un exemple et le format attendu. Elles seront évaluées séparément et ne font pas partie du critère de succès principal.

3.4 Taille et séparation

	Simple	Complexe	Non couverte	Total
Développement	60	60	60	180
Test	150	150	150	450

Table 3 – Répartition des questions (graine aléatoire 2026).

Les questions SQuAD sont séparées *par article* (30 % des articles pour le développement) afin qu'aucun paragraphe ne serve à la fois au réglage et au test. Les questions HotpotQA sont tirées sans recouvrement. Avec 150 questions par catégorie, l'intervalle de confiance à 95 % d'une proportion proche de 50 % a une demi-largeur d'environ 8 points par catégorie, et d'environ 4,6 points sur l'ensemble du test.

3.5 Format d'une question

Chaque question est une ligne JSON :

```
{"id": "hotpot:5a8b57f2...", "split": "dev", "categorie": "complexe",
 "source": "hotpotqa", "question": "...", "reponses": ["..."],
 "passages_gold": ["hotpot:3f2a...", "hotpot:9c1d..."],
 "comportement_attendu": "repondre", "type_hotpot": "bridge"}
```

3.6 Limites connues du benchmark

— Certaines questions de SQuAD sont formulées pour un paragraphe précis et deviennent floues dans un grand corpus (par exemple « What was the source of the mistake ? »). Elles pénalisent toutes les stratégies de la même manière, mais abaissent les scores absolus.

— Une question « non couverte » l'est par rapport à son paragraphe d'origine ; il est possible, quoique rare, qu'un autre passage ou la connaissance interne du modèle contienne une réponse.

— Certaines réponses de référence de HotpotQA sont bruitées (par exemple « Mary Harron (born January 12, 1953) ») ; le F1 atténue ce problème sans le supprimer.

— Les données sont en anglais et relèvent de la culture générale. La généralisation à un corpus spécialisé (QR4) devra être testée séparément.

4 Systèmes comparés

4.1 Stratégies fixes (références)

Code	Modèle	Passages	Rôle
A	Grand	0	Baseline A du cahier des charges : le modèle puissant répond seul
B	Grand	3	**Référence principale** : RAG systématique avec le grand modèle
C	Petit	0	Baseline C du cahier des charges : le petit modèle répond seul
D	Petit	3	RAG systématique avec le petit modèle

TABLE 4 – Stratégies fixes. B correspond à la configuration « tout au maximum » qu'un système non adaptatif utiliserait par défaut ; c'est la référence des critères de succès.

4.2 Références issues de la littérature (Partie III)

Conformément aux conclusions du livrable L1, deux références supplémentaires seront ajoutées :

— un **routeur fondé sur la seule question** (classifieur léger, par exemple TF-IDF + régression logistique), qui sert de contrôle pour vérifier que les signaux plus coûteux apportent réellement quelque chose ;

— une **réimplémentation simplifiée d'Adaptive-RAG**, avec un classifieur de complexité qui choisit entre 0 passage, 3 passages, ou une recherche complémentaire de 6 passages.

4.3 Espace de décision de la méthode adaptative (Partie IV)

Pour chaque question, la politique choisit une action parmi : {petit, grand} × {aucune recherche, $k = 3$, recherche complémentaire $k = 6$} × {répondre, s'abstenir}. Le coût de calcul de ses propres signaux (embeddings, scores, éventuelle génération préalable) est inclus dans la latence mesurée.

5 Protocole d'exécution

— **Prompts figés** (annexe A), identiques pour tous les systèmes, avec une consigne d'abstention explicite : répondre exactement `UNANSWERABLE` si la réponse est inconnue.

— **Génération** gloutonne (température 0), au plus 32 tokens, avec le gabarit de conversation de chaque modèle. Les sorties sont donc déterministes.

— **Modèles préchargés** en mémoire. Trois requêtes de chauffe sont exécutées et ignorées avant chaque stratégie.

— **Ordre des questions** mélangé avec une graine fixe, identique pour toutes les stratégies.

— **Latence** mesurée de bout en bout avec une horloge monotone : calcul des signaux, recherche documentaire et génération.

— **Journalisation** de chaque réponse en JSONL : sortie brute, passages utilisés, latence, tokens, note.

— **Répétitions** : les sorties étant déterministes, la qualité est mesurée une fois ; la latence de l'évaluation finale est mesurée sur 3 passages complets, et c'est la médiane par question qui est retenue.

6 Métriques

Dimension	Métrique	Définition
Qualité	F1, exact match	Normalisation officielle de SQuAD ; maximum sur les réponses de référence
Qualité	**Score global**	Moyenne sur toutes les questions : F1 si une réponse est attendue ; 1 en cas d'abstention et 0 sinon pour une question non couverte
Abstention	Couverture	Part des questions auxquelles le système répond
Abstention	Risque	Part d'erreurs parmi les réponses émises (correcte si F1 $\geq$ 0,5)
Recherche	Rappel@k	1 si tous les passages justificatifs sont parmi les k passages récupérés
Efficacité	Latence	Moyenne, médiane et 95e centile (P95), en secondes
Efficacité	Tokens	Tokens du prompt et tokens générés, par question
Efficacité	Mémoire	Pic de mémoire MLX par stratégie
Énergie	Joules / question	Puissance CPU + GPU mesurée par `powermetrics`, intégrée sur la durée de la série, après soustraction de la consommation au repos

Table 5 – Métriques du protocole.

Énergie. Conformément au cahier des charges, l'énergie n'est rapportée que si la mesure `powermetrics` a effectivement été réalisée. Sinon, latence, tokens et mémoire sont rapportés comme indicateurs indirects, sans être convertis en joules.

7 Analyse statistique

— **Intervalles de confiance** à 95 % par bootstrap (10 000 rééchantillonnages des questions).

— **Comparaisons appariées** : les différences entre systèmes sont calculées question par question, puis rééchantillonnées ensemble (bootstrap apparié), car tous les systèmes répondent aux mêmes questions.

— **Résultats par catégorie** systématiquement rapportés en plus de la moyenne globale.

— **Courbes qualité–coût** : score global en fonction de la latence moyenne, pour toutes les stratégies et pour plusieurs réglages de la méthode adaptative.

8 Critères de succès (pré-enregistrés)

Critère principal. Sur le jeu de test, par rapport à la stratégie B (grand modèle + RAG) :

(i) la latence moyenne par question est réduite d'**au moins 20 %** ;

(ii) la perte de score global est d'**au plus 2 points**, en estimation ponctuelle.

La *non-infériorité* n'est revendiquée que si la borne supérieure de l'intervalle de confiance à 95 % de la perte est elle-même inférieure à 2 points.

Critères secondaires.

— **Fiabilité (QR2)** : à couverture égale, le risque de la méthode adaptative est inférieur à celui de B ; à défaut, la méthode atteint une meilleure couverture à risque égal.

— **Contrôle (L1)** : la méthode fait mieux que le routeur fondé sur la seule question. Sinon, les signaux plus coûteux ne sont pas justifiés, et ce constat sera rapporté.

— **Robustesse (QR4)** : aucune catégorie ne perd plus de 5 points par rapport à B. Toute catégorie qui dépasse ce seuil sera signalée explicitement.

Résultat négatif. Si le critère principal n'est pas atteint, le résultat sera rapporté tel quel, avec l'analyse des catégories et des signaux responsables. Le cahier des charges rappelle qu'un résultat négatif bien établi reste informatif.

9 Pilote

Déroulement. Le pilote a exécuté les quatre stratégies fixes sur les 180 questions du jeu de *développement* (le jeu de test n'a pas été lu), en une passe, le 18 octobre 2026. Il a duré 351 s au total, chargement des modèles et calcul des embeddings du corpus compris. Les sorties brutes sont dans `results/pilote/sorties.jsonl` et le résumé dans `results/pilote/resume.json`.

Stratégie	Score	IC 95 %	Couv.	Risque	Lat. moy.	P95	Tokens
A – grand, sans RAG	0,313	[0,249 ; 0,377]	0,31	0,82	0,25 s	0,43 s	62
B – grand + RAG	**0,424**	[0,355 ; 0,493]	0,56	0,53	0,99 s	1,55 s	579
C – petit, sans RAG	0,233	[0,175 ; 0,295]	0,46	0,93	0,13 s	0,22 s	84
D – petit + RAG	0,298	[0,235 ; 0,363]	0,78	0,71	0,39 s	0,59 s	601
Oracle	*0,623*	–	–	–	*0,30 s*	–	–

TABLE 6 – Résultats du pilote (jeu de développement, 180 questions). Oracle : stratégie la moins coûteuse donnant le meilleur score pour chaque question. Couverture et risque : voir tableau 5. Pic de mémoire MLX du processus, les deux modèles étant chargés : 3,4 à 4,1 Go selon la stratégie.

Score global par catégorie	A	B	C	D
Simple	0,07	**0,45**	0,05	0,33
Complexe	0,09	0,27	0,07	**0,31**
Non couverte	**0,78**	0,55	0,58	0,25

Rappel de la recherche	@1	@3	@6	@10
Simple	0,48	0,60	0,70	0,77
Complexe	0,00	0,22	0,35	0,45

TABLE 7 – En haut : score global par catégorie (60 questions chacune). En bas : rappel de la recherche documentaire seule (pour une question complexe, les deux passages justificatifs doivent être récupérés).

Enseignements.

1. **La chaîne complète fonctionne** : construction du benchmark, recherche, génération, notation et journalisation. Les sorties sont déterministes.
2. **L'écart de coût entre stratégies est important** : B est 7,8 fois plus lente que C et consomme 7 fois plus de tokens. Le critère principal impose une latence moyenne inférieure à 0,79 s, contre 0,99 s pour B.
3. **Aucune stratégie fixe n'est la meilleure partout** : B l'emporte sur les questions simples, D sur les complexes et A sur les non couvertes. Le routeur *oracle* atteint 0,62 pour 0,30 s par question. C'est une borne théorique, inatteignable en pratique, mais elle montre une marge réelle pour une politique adaptative.
4. **L'abstention laissée au modèle est mal calibrée** : le petit modèle avec RAG répond presque toujours (couverture 0,78, risque 0,71), tandis que le grand modèle sans RAG s'abstient

sur 69 % des questions. Cela justifie de confier la décision d'abstention à la politique (QR2), plutôt qu'au seul prompt.

5. **La recherche documentaire est le maillon faible pour les questions complexes** : les deux passages justificatifs ne sont récupérés ensemble que dans 22 % des cas avec $k = 3$.
6. **Le coût de calcul est faible** : une passe complète des quatre stratégies sur le jeu de test est estimée à environ 15 minutes. Trois répétitions pour la latence sont donc réalistes.
7. **L'énergie suit les mêmes écarts que la latence** (tableau 8) : une question coûte 22,8 J avec B contre 1,9 J avec C, soit 12 fois plus. Le RAG augmente à la fois la durée et la puissance moyenne (environ 24 W contre 17 W), car le traitement du long prompt sollicite davantage le GPU.

Stratégie	Durée	Puissance moy.	Énergie nette	J / question
A – grand, sans RAG	45,5 s	17,1 W	680 J	3,8
B – grand + RAG	181,1 s	24,8 W	4 104 J	**22,8**
C – petit, sans RAG	22,9 s	16,7 W	333 J	**1,9**
D – petit + RAG	71,5 s	23,8 W	1 547 J	8,6

TABLE 8 – Énergie mesurée par `powermetrics` (puissance CPU + GPU + ANE, échantillonnée toutes les 200 ms), 18 octobre 2026, jeu de développement (180 questions). Énergie nette = énergie mesurée moins la consommation au repos (2,2 W) sur la même durée. La fenêtre de chaque stratégie inclut ses 3 requêtes de chauffe, ce qui surestime d'environ 2 % l'énergie par question. Script : `bench/energie.py` ; résultats : `results/pilote/energie.json`.

Ajustement prévu avant le gel. Seul le choix de la méthode de recherche reste ouvert. Avant de créer l'étiquette `protocole-v1.0`, on comparera *sur le jeu de développement uniquement* la recherche dense actuelle, une recherche lexicale BM25 et une combinaison des deux. Le critère retenu est le rappel@3, à coût comparable. La méthode choisie sera ensuite figée pour toutes les stratégies.

10 Gel du protocole

Avant l'évaluation finale :

1. vérifier que `data/benchmark/MANIFEST.json` (empreintes des fichiers) n'a pas changé ;
2. figer `configs/protocole_v1.json` et créer l'étiquette Git `protocole-v1.0` ;
3. développer et régler la méthode adaptative uniquement sur `dev.jsonl` ;
4. exécuter *une seule fois* toutes les méthodes sur `test.jsonl`, et publier les sorties brutes dans `results/`.

A Prompts

Sans recherche documentaire :

```
Answer the question with a short phrase only. If you do not know the answer
or it cannot be determined, reply exactly: UNANSWERABLE.

Question: {question}
Answer:
```

Avec recherche documentaire :

```
Answer the question using only the passages below, with a short phrase only.
If the passages do not contain the answer, reply exactly: UNANSWERABLE.

[1] {titre}: {texte}
[2] ...

Question: {question}
Answer:
```

B Reproduire

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python bench/build_benchmark.py          # télécharge, vérifie, construit
python bench/check_env.py                # relève l'environnement
HF_HUB_OFFLINE=1 python bench/run_pilot.py   # pilote (jeu de développement)
```

Références

Pranav Rajpurkar, Robin Jia, and Percy Liang. Know What You Don't Know : Unanswerable Questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018. URL https://arxiv.org/abs/1806.03822.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA : A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of EMNLP 2018*, 2018. URL https://arxiv.org/abs/1809.09600.